

ЛР #3: [C++ & UNIX]: C++ CLI / FUNCTION / LOOP / RECURSION

Цель

Познакомить студента с основными алгоритмическими конструкциями, которые будут использоваться для создания CLI программы. Далее продемонстрировать эффективность использования механизма рекурсии.

Задачи:

- C++ алгоритмы: CLI Калькулятор вещественных чисел +, -, ^, . Реализация с использованием только функций, условий, циклов, + и -. Вид команд в консоли: calc plus / minus / power;
- Ханойская башня, результат корректной последовательности

Решение

1. [C++ CLI CALC] Создать программу CALC с интерфейсом CLI

Создать программу под названием CALC, которая будет принимать на вход 3 аргумента (2 операнда и 1 оператор). Оператор может быть: +, -, ^. Реализация операторов только с использованием функций, условий, циклов, +, - и *.

```
#include <iostream>
#include <string>
#include <vector>
#include <bitset>

using namespace std;

int main(){
    double num1, num2;
    unsigned int power;
    double result = 1.0;
    char operation, status;
    do {
        cout << "Enter first number: ";
        cin >> num1;
        cout << "Enter operation: ";
        cin >> operation;
        switch (operation)
        {
            case '+':
                cout << "Enter second number: ";
                cin >> num2;
                cout << "Result: " << num1 + num2 << endl;
                break;
            case '-':
                cout << "Enter second number: ";
                cin >> num2;
```

```

        cout << "Result: " << num1 - num2 << endl;
        break;
    case '^':
        cout << "Enter power (must be positive integer): ";
        cin >> power;
        for (int i = 0; i < power; i++) {
            result *= num1;
        }
        cout << "Result: " << result << endl;
        break;
    default:
        cout << "No such command" << endl;
    }
    cout << "Print y or Y to continue:";
    cin >> status;
    cout << endl << endl;
} while (status == 'y' || status == 'Y');

return 0;
}

```

“” стандартные программы создания папок, см lab_1”

```

vladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_3/src$ g++ main.cpp -o main
vladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_3/src$ ./main

```

Первое число: 228
 Операция: +
 Второе число: 1337
 Result: 1565
 Введите y/Y для повтора:y

Первое число: 69
 Операция: -
 Второе число: 28
 Result: 41
 Введите y/Y для повтора:y

Первое число: 5
 Операция: 4
 Ошибка ввода
 Введите y/Y для повтора:y

Первое число: 5
 Операция: ^

Второе число (положительное, целое): 4

Result: 625

Введите у/У для повтора:у

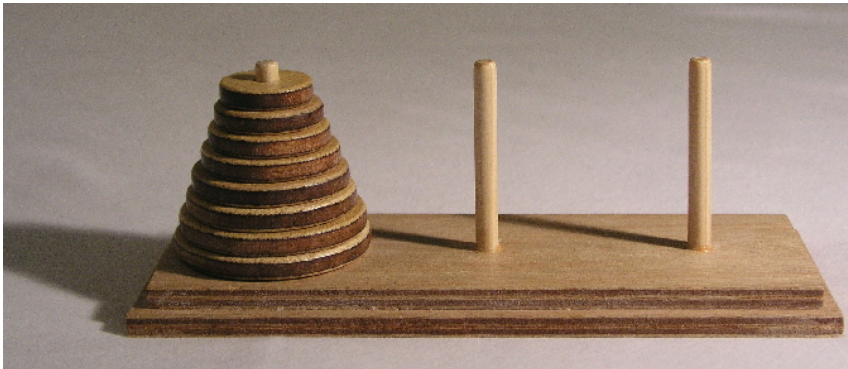
Первое число: 67

Операция: 67

Ошибка ввода

Введите у/У для повтора:

2. [C++ RECURSION] Решить задачу ханойской башни с использованием рекурсии



Описание: Ханойская башня является одной из популярных головоломок XIX века. Даны три стержня, на один из которых нанизаны восемь колец, причём кольца отличаются размером и лежат меньшее на большем. Задача состоит в том, чтобы перенести пирамиду из восьми колец за наименьшее число ходов на другой стержень. За один раз разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее.

Результат обнаруженной последовательности шагов записать в виде двусвязного списка. В конце программы сделать вывод этого списка на экран. Освободить память списка перед завершением программы.

```
#include <iostream>
#include <list>
#include <string>

using namespace std;

void hanoi_towers(int quantity, int from, int to, int buf_peg,
list<string>& listOfOperations)
{
    if (quantity != 0)
    {
        hanoi_towers(quantity-1, from, buf_peg, to,
listOfOperations);

        string move = to_string(from) + "->" + to_string(to);
```

```

        listOfOperations.push_back(move);

        hanoi_towers(quantity-1, buf_peg, to, from,
listOfOperations);
    }
}

int main()
{
    list<string> listOfOperations;
    setlocale(LC_ALL, "rus");
    int start_peg, destination_peg, buffer_peg, plate_quantity;
    cout << "Номер первого столбика:" << endl;
    cin >> start_peg;
    cout << "Номер конечного столбика:" << endl;
    cin >> destination_peg;
    cout << "Номер промежуточного столбика:" << endl;
    cin >> buffer_peg;
    cout << "Количество дисков:" << endl;
    cin >> plate_quantity;
    hanoi_towers(plate_quantity, start_peg, destination_peg,
buffer_peg, listOfOperations);

    for (const string& op : listOfOperations) {
        cout << op << endl;
    }

    cout << "Пустой ли список до очистки: " << boolalpha <<
listOfOperations.empty() << endl;
    listOfOperations.clear();
    cout << "Пустой ли список после очистки: " << boolalpha <<
listOfOperations.empty() << endl;
    return 0;
}

```

```

vladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_3/src$ g++
hanoi_tower.cpp -o main
vladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_3/src$ ./main

```

```

Номер первого столбика:
1
Номер конечного столбика:
2
Номер промежуточного столбика:

```

3

Количество дисков:

4

1->3

1->2

3->2

1->3

2->1

2->3

1->3

1->2

3->2

3->1

2->1

3->2

1->3

1->2

3->2

Пустой ли список до очистки: false

Пустой ли список после очистки: true

3. [SAVE] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).